EECE 2323 Digital Logic Design Lab

Name: Arefin Mahmood

Lab Report 1: 16-bit Adder
Date: 05/20/2026

Background & Purpose

In this lab, I designed a 16-bit signed two's complement adder in SystemVerilog, simulated it in Vivado, and then tested it on the TUL PYNQ board. The main goal was to build a circuit that takes two 16-bit inputs, here it was input a and input b, and add them together, and it sends the result to the 16-bit output, here it's f. The circuit also includes an overflow flag, called ovf, which turns on when the signed answer is too large or too small to fit correctly in 16 bits. This matters because the lab is using signed two's complement arithmetic, so the important warning signal is overflow, and not a carry flag.

This lab connects directly to how processors handle arithmetic. Adders are used in ALUs, datapaths, address calculations, and many instruction operations. Even though this design is only a 16-bit adder, the same basic idea appears in larger processor circuits. One important part of the lab was seeing that a binary output can look valid as bits, but still represent the wrong signed value if overflow happens. By writing the design, checking it in simulation, and then testing it on real hardware, I was able to connect signed binary addition to an actual circuit implementation.

Design Analysis

In Lab 0, the focus was on a smaller full adder circuit so the design only dealt with adding individual bits. But in this lab, that idea was expanded into a wider datapath by building a 16-bit signed adder. This new design takes two 16-bit inputs a and b, and sends the 16-bit sum to f. Because the inputs are signed two's complement numbers, the circuit also needs the overflow output, ovf, to show when the signed result does not fit correctly in 16 bits.

For the prelab and simulation, I used several input combinations to check both regular addition and overflow behavior. The main sixteenbit_adder module did not need major changes before the hardware test because the behavioral SystemVerilog code already matched the expected simulation results. The hardware version added the provided top module, adaptor display module, constraint file, and VIO core around my adder. The VIO core was necessary because the board does not have enough physical switches to enter both 16-bit inputs at the same time. With VIO, I could type the values for a and b from Vivado, observe the output, and compare the hardware behavior to the simulation waveform.

Results and Analysis

The simulation confirmed that the 16-bit adder was producing the expected values for the test cases I chose. As shown in Figure A1 and Table A1 in Appendix A, the normal addition cases behaved correctly. One example was 16'h0025 + 16'h0045, which gave 16'h006A with ovf = 0. The overflow cases also matched what should happen in signed two's complement arithmetic. For positive overflow, 16'h7FFF + 16'h0001 gave 16'h8000 with ovf = 1, since two positive inputs wrapped into a negative result. For negative overflow, 16'hA415 + 16'hA555 gave 16'h496A with ovf = 1, since two negative inputs produced a positive result. From these results, the overflow logic was working as intended because it only turned on when the signed result could not be represented correctly in 16 bits.

After the simulation passed, I tested the design on the TUL PYNQ board using the VIO

dashboard. I entered the values for a and b through the VIO output probes, then checked f and ovf through the VIO input probes. The hardware results matched the simulation. In Figure C1, the non-overflow test produced 16'h006A with ovf = 0. In Figure C2, the overflow test produced 16'h394F with ovf = 1. During the lab, I ran into a Vivado implementation/checkpoint issue while using an older project setup. Restarting the project with a cleaner file path and regenerating the bitstream fixed the problem. After that, the board programmed successfully, and the hardware output matched the expected results.

Conclusion & Recommendations

This lab showed me how a 16-bit signed two's complement adder is built, checked in simulation, and then tested on actual hardware using SystemVerilog and Vivado. From the simulation waveform and the VIO hardware tests, the adder produced the expected 16-bit outputs and detected signed overflow correctly. The main point I took from this lab is that signed overflow is different from carry out. A carry out only tells part of the story for unsigned math, while signed overflow depends on the sign bits of a, b, and f.

The results showed that overflow should turn on when the signs stop making sense for signed addition. If two positive inputs produce a negative result, or if two negative inputs produce a positive result, then the signed result has gone outside the 16-bit two's complement range. For future testing, I would recommend using a mix of test values instead of only simple addition cases. The test set should include normal positive addition, normal negative addition, mixed-sign addition, positive overflow, and negative overflow. That gives a better check that the adder works correctly across the main signed two's complement situations.

Appendix A: Prelab and Test Data/Simulation

a	b	f	ovf
16'h0025	16'h0045	16'h006A	0
16'hA415	16'hA555	16'h496A	1
16'hF115	16'hF215	16'hE32A	0
16'h9D00	16'h9E00	16'h3B00	1
16'hED00	16'hEF03	16'hDC03	0
16'h8A10	16'h7110	16'hFB20	0
16'h21AA	16'h21BB	16'h4365	0
16'h9A00	16'h9F4F	16'h394F	1

Table A1: Prelab Test Cases

Overflow Equation

ovf = (~(a[15] ^ b[15])) & (a[15] ^ f[15])

This equation checks whether the two inputs have the same sign, but the output has the opposite sign.

Additional Test Case

a	b	f	ovf
16'h0040	16'h0050	16'h0090	0

This testcase was added to help test missing bit cases from the original prelab table.

Test Type	a	b	Expected f	Expected ovf
Normal addition	16'h0025	16'h0045	16'h006A	0
Positive overflow	16'h7FFF	16'h0001	16'h8000	1
Negative overflow	16'hA415	16'hA555	16'h496A	1
Negative addition	16'hF115	16'hF215	16'hE32A	0
Negative overflow	16'h9D00	16'h9E00	16'h3B00	1
Negative addition	16'hED00	16'hEF03	16'hDC03	0
Mixed-sign addition	16'h8A10	16'h7110	16'hFB20	0
Positive addition	16'h21AA	16'h21BB	16'h4365	0
Negative overflow	16'h9A00	16'h9F4F	16'h394F	1
Extra bit test	16'h0040	16'h0050	16'h0090	0

Table A2: Simulation Test Cases

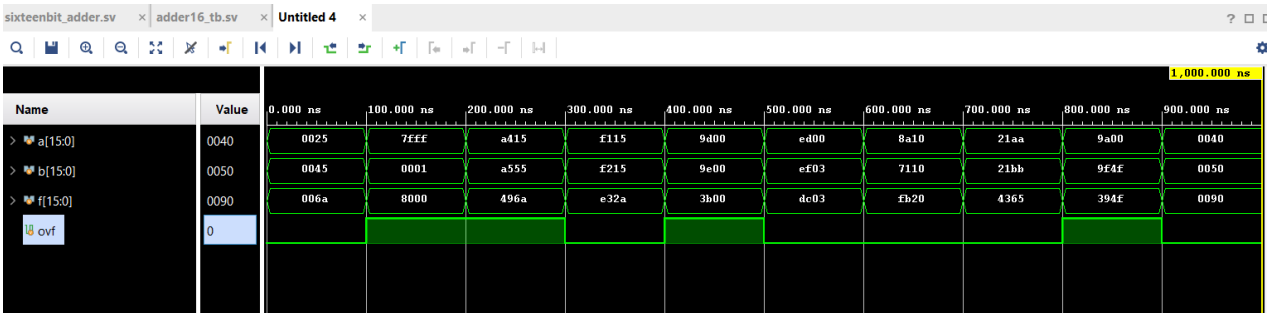


Figure A1: Behavioral Simulation Waveform for 16-bit Adder

Figure A1 shows the behavioral simulation waveform for the 16-bit adder testbench. The waveform includes normal addition, positive overflow, negative overflow, mixed-sign addition, and the extra testcase.

Appendix B: Design Program Files

Figure B1: sixteenbit_adder.sv Code

```
module sixteenbit_adder(
    input  logic [15:0] a,
    input  logic [15:0] b,
    output logic [15:0] f,
    output logic        ovf
);
    assign f = a + b;
    assign ovf = (~(a[15] ^ b[15])) & (a[15] ^ f[15]);
endmodule
```

Figure B1 shows the SystemVerilog code for the 16-bit signed adder. The design adds inputs a and b to produce f, and the overflow equation checks the sign bits to determine ovf.

Figure B2: adder16_tb.sv Testbench Code

```
`timescale 1ns / 1ps
module adder16_tb;
    logic [15:0] a;
    logic [15:0] b;
    logic [15:0] f;
    logic        ovf;

    sixteenbit_adder dut (
        .a(a),
        .b(b),
```

```
        .f(f),
        .ovf(ovf)
    );

    initial begin
        a = 16'h0025; b = 16'h0045;
        #100 a = 16'h7FFF; b = 16'h0001;
        #100 a = 16'hA415; b = 16'hA555;
        #100 a = 16'hF115; b = 16'hF215;
        #100 a = 16'h9D00; b = 16'h9E00;
        #100 a = 16'hED00; b = 16'hEF03;
        #100 a = 16'h8A10; b = 16'h7110;
        #100 a = 16'h21AA; b = 16'h21BB;
        #100 a = 16'h9A00; b = 16'h9F4F;
        #100 a = 16'h0040; b = 16'h0050;
        #100 $finish;
    end
endmodule
```

Figure B2 shows the testbench used to simulate the 16-bit adder. The testbench applies multiple input values to check normal addition, positive overflow, negative overflow, mixed-sign addition, and the extra testcase.

Appendix C: Output Screen Captures

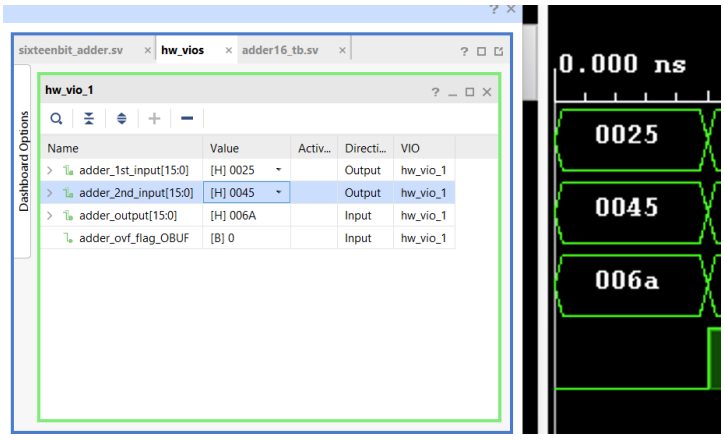


Figure C1: VIO Hardware Test for Non-Overflow Case

Figure C1 shows the hardware test for a normal addition case. The inputs a = 16'h0025 and b = 16'h0045 produced f = 16'h006A, and the overflow flag stayed at 0.

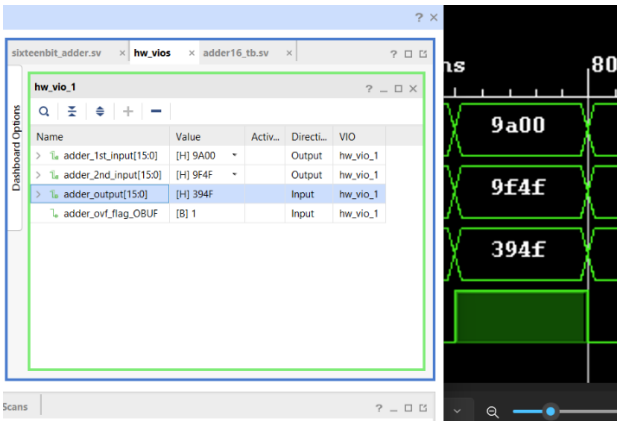


Figure C2: VIO Hardware Test for Overflow Case

Figure C2 shows the hardware test for a signed overflow case. The inputs a = 16'h9A00 and b = 16'h9F4F produced f = 16'h394F, and the overflow flag turned on because two negative signed inputs produced a positive signed output.

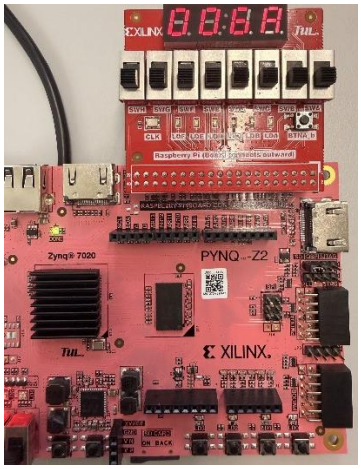


Figure C3: Board Output for Non-Overflow Case

Figure C3 shows the TUL PYNQ board during a non-overflow test case. The LD3 overflow LED stayed off, which matched the expected ovf = 0 result.

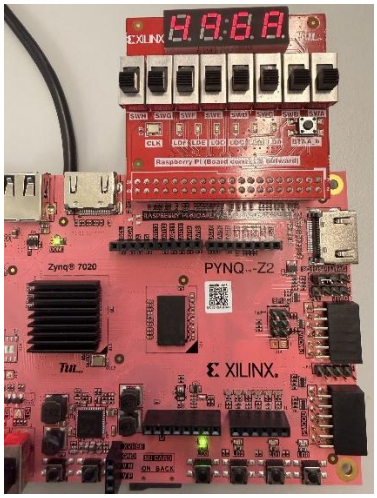


Figure C4: Board Output for Overflow Case

Figure C4 shows the TUL PYNQ board during an overflow test case. The LD3 overflow LED turned on, which matched the expected ovf = 1 result.